

REPORT

ASSIGNMENT 5

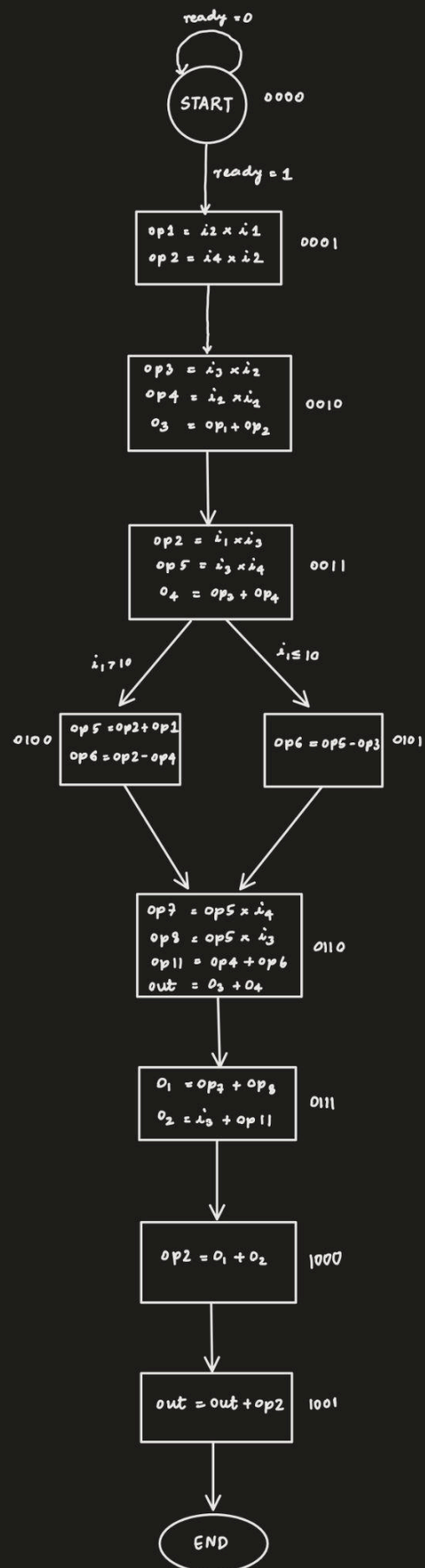
ANUSHKA GUPTA	220101014
KASHISH AGGARWAL	220101050
MAHEK CHOUDHARY	220101062
MAHUA SINGH	220101063

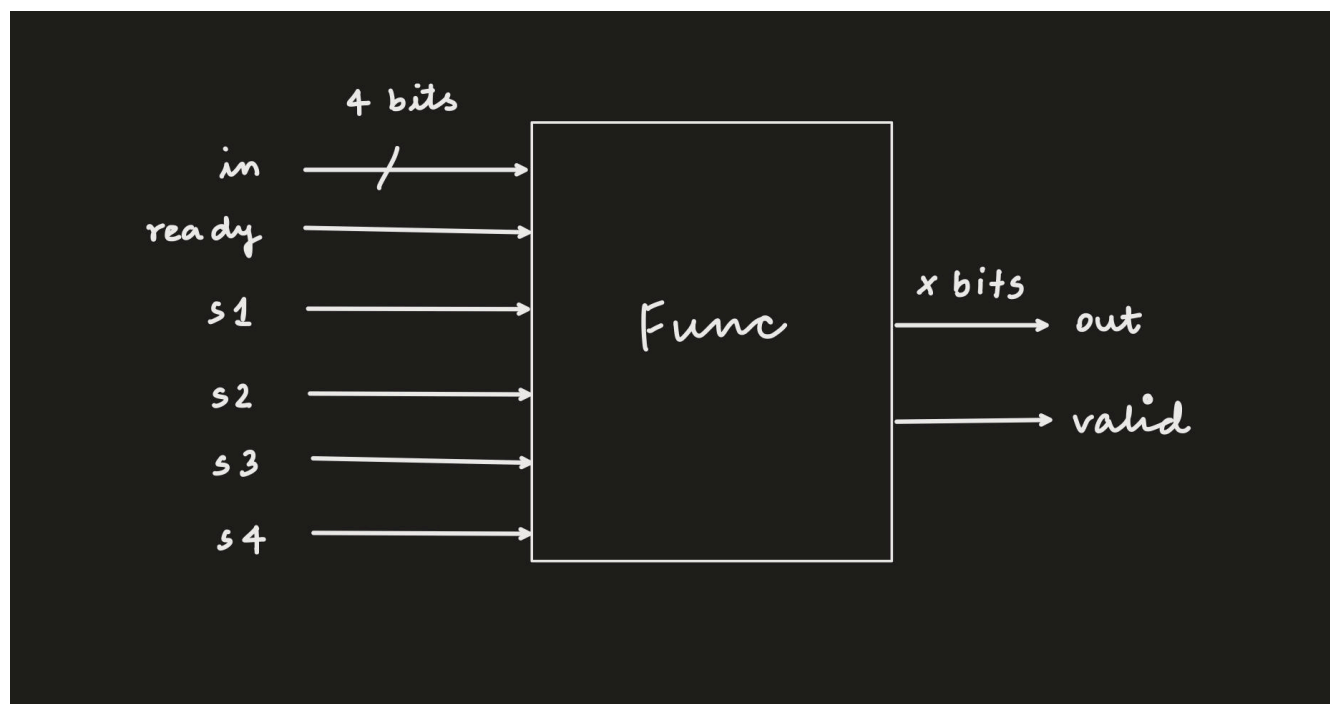
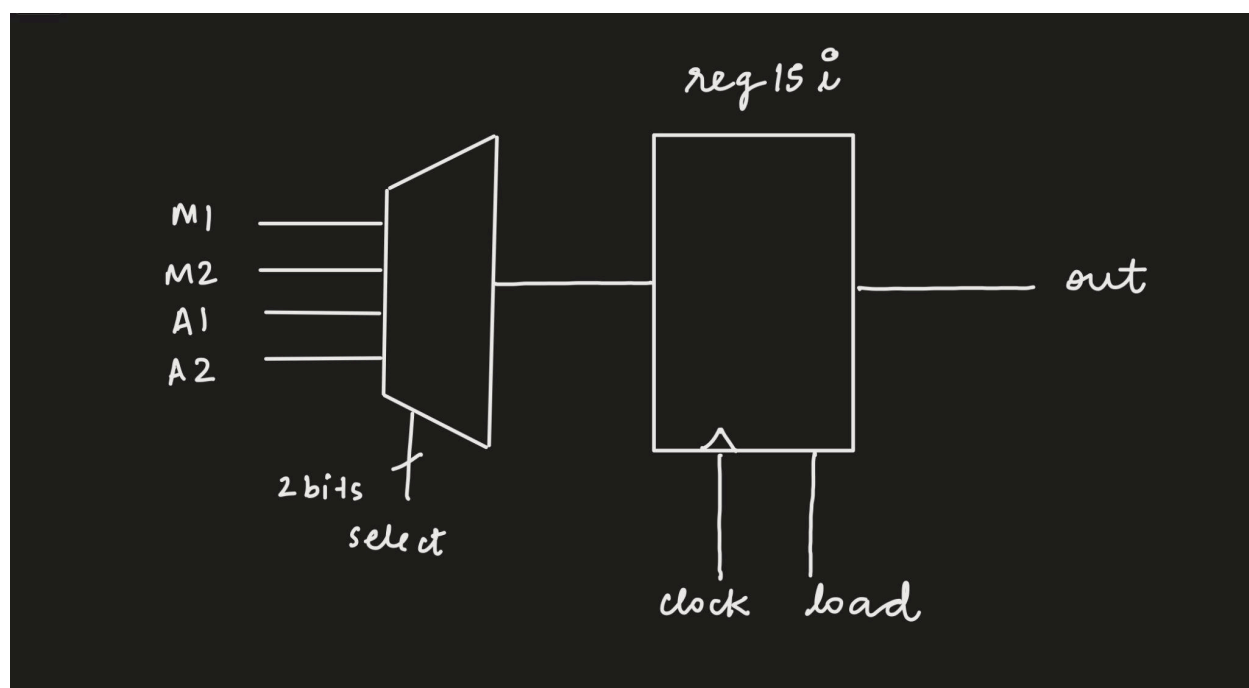
LOGIC BEHIND CIRCUIT

Finite State Machine (FSM):

The heart of the system is the FSM, which orchestrates the sequence of operations based on inputs and internal states. Here's how it operates:

- **State Transition** : The FSM transitions between different states, each representing a specific stage of the computation process. Transitions are triggered by clock pulses and depend on various input conditions.
- **Operation Control** : In each state, the FSM controls which operations are enabled or disabled. It activates or deactivates the enable signals for arithmetic operations, data storage, and comparison based on the current state and input conditions.
- **Data Flow Management** : The FSM manages the flow of data between different modules. It selects the appropriate inputs for arithmetic operations, decides which data to store in registers, and determines when to perform comparisons.





Arithmetic and Comparison Modules

The FSM interacts with several modules to perform arithmetic operations (addition, subtraction, multiplication) and comparisons:

- **Adder-Subtractor Module** : Performs addition or subtraction of two operands based on a control signal. It handles both addition and subtraction operations efficiently by exploiting the value of “**invert**”.
- **Multiplier Module** : Computes the product of two operands using simple multiplication logic. It efficiently calculates the multiplication result.
- **Comparison Module (comp)** : Determines whether one operand is less than the other. It outputs a signal indicating the result of the comparison.

Multiplexers and Registers

Multiplexers (MUX) and registers play a crucial role in data selection and storage:

- **MUX Modules** : Select one of multiple input data based on control signals. They facilitate data routing and selection within the system.
- **Register Modules** : Store data temporarily and allow for selective loading based on control signals. They enable the system to hold intermediate results and input data for further processing.

Overall Operation:

Here's how the system operates as a whole:

- 1. Initialization** : The system starts in an initial state where all operations are disabled, awaiting input signals.
- 2. State Transition** : Upon receiving input signals (e.g., `ready` signal), the FSM transitions to the next state, enabling specific operations and determining the flow of data.
- 3. Arithmetic and Comparison** : In each state, the enabled operations (such as addition, subtraction, multiplication, and comparison) are performed based on the input data and control signals.
- 4. Data Storage** : Intermediate and final results are stored in registers for further processing or output.
- 5. State Advancement** : After completing the current state's operations, the FSM transitions to the next state based on the current state and input conditions.
- 6. Completion and Output** : The system continues through the state transitions until the computation is complete. Final results are outputted when the computation is finished.

code.v testbench.v inputOutput.xdc Untitled 10

340,000 ns

Timing diagram showing signals: clk, ready, s1, s2, s3, s4, in[3:0], out[...0], and valid over 340,000 ns.

Signal	Value
clk	1
ready	1
s1	0
s2	0
s3	0
s4	1
in[3:0]	1
out[...0]	8
valid	1